

Team Task Division

Urban Heat & Tree-Canopy Priority System — Team Task Division

Team size: 5 | **Project window:** Sept 2025 – April 2026 | **City:** Pune
(fallback: Bengaluru)

This document splits the full project scope into 5 independent workstreams so each person can start immediately without waiting on the others. Each section covers: what the role does, the person's end goal, a detailed step-by-step plan, what "done" looks like, and how much of the overall project it covers.

Person 1 — Geo Pipeline Lead (Boundaries · LST · Data-Centre Buffers)

What this role does

Owns everything geospatial that isn't the neural network: getting clean ward boundaries, computing surface temperature from satellite data, and running the data-centre heat-contribution analysis — the project's headline finding.

Goal

Produce a `wards.geojson` file where every ward has a computed mean surface temperature, and a separate analysis showing how much hotter/less green wards near data centres are compared to wards farther away.

Step-by-step

1. **Register Earth Engine access first.** Create a Google Cloud project, enable the Earth Engine API, run `earthengine authenticate`. This can take a day or two for approval — start it before anything else.

2. Resolve ward boundaries.

- Search the Pune Municipal Corporation open-data portal, the Pune Smart City data portal, and OpenStreetMap (via `osmnx`, querying admin boundaries) for ward-level shapefiles.
- Load candidates with `geopandas.read_file()` and check: all ~40+ wards present, no gaps or overlaps, consistent ward IDs/names.
- **Decision point:** if Pune's boundaries are clean, commit to Pune. If not, repeat the search for Bengaluru's BBMP wards. Don't spend more than 2 days here — this is the single highest-risk item in the whole project.
- Output: `data/wards.geojson`.

3. Pull and mask Landsat imagery.

```
landsat = (ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
           .filterBounds(city_boundary)
           .filterDate('2024-03-01', '2024-05-31') # hot season
           .filter(ee.Filter.lt('CLOUD_COVER', 20)))
```

4. Compute land surface temperature.

- Convert the thermal band (ST_B10) to Celsius using Collection 2 scaling factors.
- Compute NDVI-derived emissivity (proportion-of-vegetation method) and apply the emissivity correction to get true LST (not just brightness temperature).
- Export as GeoTIFF via `ee.batch.Export.image.toDrive()`.

5. Aggregate LST to wards.

- Use `rasterio` + `geopandas` (or `geemap.zonal_statistics`) to compute mean LST per ward.
- Add an `lst_mean` column to `wards.geojson`.

6. Map data-centre locations.

- Compile known data-centre sites in the city from public listings, news coverage, and OSM tags.
- Buffer each location at 500m / 1km / 2km using building-footprint data (OSM or Microsoft footprints).

7. Run the inside-vs-outside comparison.

- Zonal-stat LST and canopy (once available from Person 2) separately for pixels inside vs. outside each buffer ring.
- Produce the quotable finding: “data-centre-adjacent wards show X°C higher mean LST and Y% lower canopy.”

8. Export everything to the shared /data folder as GeoJSON/CSV for the rest of the team.

What “done” looks like

- wards.geojson with a populated lst_mean column.
- A data-centre buffer layer plus one chart/table showing the inside-vs-outside LST and canopy comparison.
- A short written note on which city was chosen and why.

Project coverage

~20% — covers the geo-pipeline half of Section 4, all of Section 7 (boundary risk), and Sections 8.1–8.2 and 8.9 of the project spec.

Person 2 — ML Engineer (Land-Cover Segmentation / U-Net)

What this role does

Owns the project's one neural network — a U-Net that tells real tree canopy apart from lawns, farms, and playing fields, which raw NDVI can't do. This role is technically self-contained: it only needs Sentinel-2 imagery, which can be pulled independently of Person 1.

Goal

Produce a per-ward “% true canopy cover” number that measurably beats a naive NDVI-threshold baseline, with reported accuracy metrics.

Step-by-step

1. **Pull Sentinel-2 tiles** for the same city and study window via Earth Engine (10m resolution, multi-band).
2. **Pull training labels** from ESA WorldCover or Dynamic World land-cover products for the same area.
3. **Build the naive baseline first** (do this early — it’s cheap and other people may need it before your model is ready):
 - Threshold NDVI (e.g., > 0.4) as “green.”
 - Compute % green per ward. This is what your model has to beat.
4. **Design and train the U-Net.**
 - Classes: built-up / tree-canopy / grass-&-crop / water / bare.
 - Start from a pretrained encoder backbone if time is tight (faster convergence, less data needed).
 - Train on free Colab or Kaggle GPU.
5. **Merge in the pretrained canopy-height layer** (Meta/WRI ~1m Canopy Height Maps) as a second, independent tree-detection signal — this sharpens the tree class beyond what Sentinel-2’s 10m pixels can resolve alone.
6. **Evaluate honestly.**
 - Report per-class IoU/F1, especially for the canopy class specifically.
 - Directly compare U-Net accuracy vs. the naive NDVI baseline — this comparison is the most examinable part of the project, so keep the numbers clean and reproducible.
7. **Aggregate to per-ward canopy %** and export as CSV (`ward_id`, `canopy_pct`).

What “done” looks like

- A trained model checkpoint.
- A CSV of per-ward canopy percentage.
- A table: NDVI-baseline accuracy vs. U-Net accuracy, per class.

Project coverage

~20% — covers Section 3 (the core ML contribution) and Section 8.1 (training) of the project spec. This is the most self-contained workstream — it can run almost entirely in parallel with everyone else.

Person 3 — Backend & Data Engineer (FastAPI · Firebase · Scoring)

What this role does

Turns Person 1's and Person 2's raw outputs into a served API, and owns the two pieces of “policy-ready” logic: the priority score and the cost/impact estimator. Can start immediately with dummy data — doesn't need to wait for real geo/ML outputs to begin.

Goal

A running FastAPI server that serves real, scored, per-ward data, plus a working Firebase-backed report-save flow.

Step-by-step

1. **Set up Firebase.** Create the project, enable Auth, Firestore, and Storage. Define the schema:

```
users/{uid}
├─ displayName, email, role, lang
└─ reports/{reportId}
    ├─ city, ward, mode, createdAt
    ├─ snapshot: { lst, canopyPct, priority, vulnerability }
    ├─ actionPlan: { trees, costINR, coolingC, co2t }
    └─ files: { pdfUrl, mapPngUrl, geojsonUrl }
```

2. **Build the FastAPI skeleton early, against mock data:**

- GET /cities
- GET /wards/{city}
- GET /wards/{city}/{id}

3. **Pull Census 2011 ward-level data** and compute a normalized vulnerability score (density + relevant census indicators).

4. Once Person 1's LST/DC data and Person 2's canopy data arrive, write the priority-score function:

```
Priority = f(high LST, low canopy, high vulnerability)
```

Normalize and rank all wards.

5. Build the cost/impact estimator.

- Compile a lookup table of published per-tree cost and CO₂-sequestration coefficients.
- Apply it to each ward's canopy gap to produce: trees needed, estimated cost (₹), estimated 5-year cooling (°C), estimated CO₂ offset.
- State clearly in code comments/docs that these are estimates from published coefficients, not field measurements.

6. Build the data-centre endpoint: `GET /datacentres/{city}` serving Person 1's buffer analysis.

7. Build the report endpoints:

- `POST /reports` — generate + store a report, return its ID and file URLs.
- `GET /reports` — the signed-in user's report history (auth-gated).

What “done” looks like

- `/wards/pune` returns real, scored ward data (not mock).
- A working report-save flow: hitting `POST /reports` creates a Firestore record with a snapshot.

Project coverage

~25% — covers Section 6 (backend/data model) and Sections 2.1/2.3 (scoring and cost estimator) of the project spec. This role is the integration hub: starts earliest (with dummy data) and finishes latest (once real data lands).

Person 4 — Frontend: Citizen View

What this role does

Builds the public-facing app: search, heat map, ward detail, action plan, and report download. Can build against Person 3's mock endpoints from day one — no need to wait for real data.

Goal

A clickable demo where someone can sign in, search a ward, see its heat/canopy summary, and download an action-plan report.

Step-by-step

1. **Scaffold the app:** `npx create-react-app` , install `leaflet` / `react-leaflet` (or Mapbox GL).
2. **Build S1 (splash/landing)** — project name, one-line value prop, “Explore as Citizen” / “Sign in as Planner” buttons.
3. **Build S2/S3 (auth + role select)** using the Firebase Auth SDK (email + Google sign-in). Citizens should be able to browse without an account.
4. **Build S4 (city/ward search)** with autocomplete on ward name/number.
5. **Build S5 (heat-map home)** — the main screen:
 - Color-coded choropleth (red = urgent, green = healthy).
 - Legend and a layer toggle (Heat / Canopy / Priority).
 - A sidebar list of “most urgent wards.”
6. **Build S6 (ward detail card)** — priority rank, mean LST, % canopy, vulnerability note, recommended native species (pulled from `/wards/{city}/{id}`).
7. **Build S7 (action plan)** — trees needed, estimated cost, estimated cooling/CO₂ benefit, each figure clearly marked as an estimate with its source.
8. **Build S8 (download/report confirmation)** — wired to `POST /reports` , offering share/download once generated.

What “done” looks like

A working flow: sign in → search a ward → view its card and action plan → generate and download a report.

Project coverage

~20% — covers Sections 5.1–5.2 (citizen flow) and 5.4 (report history, shared with Person 5 if time is tight) of the project spec.

Person 5 — Frontend: Planner View + Integration/QA/Demo

What this role does

Builds the scientist/planner-facing screens, and additionally owns cross-team integration, testing, and the final demo — since the planner screens naturally need everyone else’s outputs, this role becomes the project’s glue.

Goal

A fully working planner dashboard on real data, plus a merged, demo-ready application and the written methodology.

Step-by-step

1. **Build S9 (analyst dashboard)** — the same map as the citizen view but with independent layer toggles (LST / NDVI / U-Net land-cover / canopy-height / vulnerability), opacity sliders, basemap switch.
2. **Build S10 (ward analytics)** — per-ward charts: 5-year LST trend, canopy trend, and a breakdown showing how much each factor (heat/canopy/vulnerability) contributed to the priority score. Show confidence values, don’t hide them.
3. **Build S11 (data-centre analysis screen)** — directly consumes Person 1’s buffer output: DC locations with distance rings, side-by-side LST/canopy comparison, the headline X°C/Y% finding.
4. **Build S12 (export)** — GeoJSON/CSV export for QGIS, plus a methodology panel documenting data sources, the emissivity correction, index formulas, and stated assumptions.

5. **Own weekly integration.** Merge everyone's branches, keep the / data file contracts consistent across all four other roles, catch mismatches early.
6. **Own the write-up and demo.** Draft the October-gate note, the eventual final report, and rehearse the live demo end-to-end.

What “done” looks like

The full planner dashboard working on real (not mock) data, a cleanly merged app, and a written methodology document ready for submission.

Project coverage

~15% direct build (Sections 5.3, 5.4 partial) — plus the integration work that makes the other four workstreams' outputs actually function as one system.

What Gets Completed Overall

If all five roles hit their individual “done” states, the completed system covers:

Area	Status after all 5 roles complete their steps
Ward boundaries + LST	✓ Done (Person 1)
Data-centre buffer analysis	✓ Done (Person 1)
U-Net canopy segmentation	✓ Done (Person 2)
Priority score + cost/impact estimator	✓ Done (Person 3)
Backend API + Firebase report history	✓ Done (Person 3)
Citizen-facing app (S1–S8)	✓ Done (Person 4)
Planner-facing app (S9–S12)	✓ Done (Person 5)
Integration + methodology doc	✓ Done (Person 5)
5-year prediction layer (stretch goal)	✗ Not covered — explicitly deferred in project scope
Hindi language toggle (S15)	✗ Not covered unless added as a stretch task

In short: completing all five workstreams covers the full “Target” tier of the project as originally scoped — everything short of the optional stretch prediction layer and the language toggle. That’s roughly **100% of the minimum + target scope**, leaving only the explicitly optional stretch items (Section 3.3 and S15) undone.

Sequencing note

Persons 1 and 2 are the critical path for the near-term feasibility gate — their outputs (ward boundaries, LST, naive canopy baseline) are what everything else eventually depends on. Persons 3, 4, and 5 should build their skeletons against mock/dummy data in parallel so there’s no idle time waiting on real geo/ML outputs to arrive.